handling when a timeout expires, including providing the user with the correct status about the operation's outcome.

When Catalog Service is overloaded, it can take several seconds to get a JDBC connection from the pool, fetch data from the database, and send a response back to Order Service. In that case, you could think of retrying the request rather than falling back on a default behavior or throwing an exception.

8.4.2 Retries

When a service downstream doesn't respond within a specific time limit or replies with a server error related to its momentary inability to process the request, you can configure your client to try again. When a service doesn't respond correctly, it's likely because it's going through some issues, and it's unlikely that it will manage to recover immediately. Starting a sequence of retry attempts, one after the other, risks making the system even more unstable. You don't want to launch a DoS attack on your own applications!

A better approach is using an *exponential backoff* strategy to perform each retry attempt with a growing delay. By waiting for more and more time between one attempt and the next, you're more likely to give the backing service time to recover and become responsive again. The strategy for computing the delay can be configured.

In this section, you'll configure retries for `BookClient`. Figure 8.8 details how the request/response interaction will work when retries are configured with exponential backoff. For example, the figure shows a scenario where each retry attempt's delay is computed as the number of attempts multiplied by 100 ms (the initial backoff value).

